

Ponteiros para Funções

Em C, ponteiros para função permitem que funções sejam tratadas como valores: elas podem ser armazenadas em variáveis, passadas como parâmetros e chamadas indiretamente ao longo da execução do programa. Esse recurso é bastante útil quando queremos escrever funções mais genéricas, capazes de executar comportamentos diferentes dependendo da função recebida.

EXERCÍCIO - Playlist

Seus amigos estão organizando uma festa e decidiram que a música é importante demais para ser escolhida de qualquer jeito. Como você é a pessoa do grupo que sabe programar, foi escolhido para desenvolver um sistema capaz de gerenciar a playlist oficial da festa.

A playlist deverá armazenar músicas com informações como nome, artista, álbum e gênero. Além de adicionar e remover músicas, o sistema também deverá permitir visualizar a playlist completa ou filtrar as músicas por artista, álbum ou gênero. Assim, será possível responder rapidamente perguntas como: “quais músicas da Ariana Grande estão na playlist?”, “quais músicas desse álbum foram adicionadas?” ou “quais músicas de Pop vão tocar?”.

Para evitar repetição de código, essas diferentes formas de busca e impressão deverão ser implementadas com o uso de ponteiros para função. Em C, ponteiros para função permitem que funções sejam passadas como parâmetros, possibilitando que uma mesma função genérica execute comportamentos diferentes dependendo da função recebida.

Neste exercício, você deverá implementar um sistema de gerenciamento de playlist utilizando dois TADs já especificados: `Musica` e `Playlist`.

O foco principal da atividade é praticar o uso de ponteiros para função, especialmente na implementação de funções genéricas de comparação e impressão.

Os arquivos `musica.h` e `playlist.h` serão disponibilizados e não devem ser alterados. Eles contêm as definições das estruturas, constantes, tipos de ponteiros para

função e assinaturas das funções que deverão ser implementadas.

Você deverá desenvolver os seguintes arquivos:

musica.c
playlist.c
main.c

TAD Música

O TAD Música representa uma única música da playlist.

Entre as funcionalidades esperadas estão:

- criar uma música;
- ler os dados de uma música;
- comparar duas músicas;
- comparar o nome de uma música com uma string;
- comparar o artista de uma música com uma string;
- comparar o álbum de uma música com uma string;
- comparar o gênero de uma música com uma string;
- imprimir uma música em diferentes formatos;
- desalocar a memória utilizada por uma música.

As assinaturas completas das funções, bem como os formatos esperados de impressão, estão descritos no arquivo `musica.h`.

TAD Playlist

Entre as funcionalidades esperadas estão:

- criar uma playlist;
- adicionar músicas à playlist;
- remover músicas da playlist;
- imprimir músicas da playlist;
- realizar impressões filtradas;
- desalocar a memória utilizada pela playlist.

As assinaturas completas das funções estão descritas no arquivo `playlist.h`.

Ponteiros para função

Um dos principais objetivos deste exercício é trabalhar o uso de **ponteiros para função**. No arquivo `musica.h`, são definidos tipos para funções de comparação e de

impressão:

```
typedef void (*tipoImpressaoMusica)(Musica *);  
typedef int (*compara)(Musica *, char *);
```

Esses ponteiros devem ser utilizados para permitir que uma mesma função da playlist consiga realizar diferentes tipos de comparação e impressão.

A função genérica responsável pelas impressões da playlist é:

```
void impressFunction(Playlist *playlist, char *key, compara comparador,  
tipoImpressaoMusica impMus);
```

Todas as impressões da playlist devem ser feitas utilizando essa função. Observe que ponteiros para função podem receber o valor NULL, quando não for necessário aplicar determinada comparação ou filtragem.

Funcionamento da entrada

Na primeira linha da entrada, será informado o nome da playlist. Na linha seguinte, será informada a quantidade de ações que serão executadas sobre essa playlist. Depois disso, para cada ação, será informado um número correspondente ao comando desejado.

Comandos do programa

O programa deverá reconhecer os seguintes comandos:

- 1 - Inserir música
- 2 - Remover música
- 3 - Imprimir playlist
- 4 - Imprimir músicas de um artista
- 5 - Imprimir músicas de um álbum
- 6 - Imprimir músicas de um gênero

Inserir Música - Ao receber o comando 1, o programa deverá ler os dados de uma nova música e inseri-la na playlist. A leitura de uma música deve seguir o seguinte formato:

nome
artista

album
genero

Exemplo:

sweetener
Ariana Grande
sweetener
Pop

Remover música - O programa deverá ler o nome de uma música e tentar removê-la da playlist. Caso a música exista na playlist, ela deverá ser removida. Caso contrário, a remoção não deverá ocorrer.

Imprimir Playlist - o programa deverá imprimir todas as músicas presentes na playlist.

Imprimir músicas de um artista - O programa deverá imprimir todas as músicas de um artista pelo nome.

Imprimir músicas de um álbum - O programa deverá ler o nome de um álbum e imprimir todas as músicas desse álbum que estão na playlist.

Imprimir músicas de um gênero - o programa deverá ler o nome de um gênero e imprimir todas as músicas desse gênero que estão na playlist.

As seguintes regras devem ser respeitadas durante a implementação:

Playlist cheia

Caso o usuário tente inserir uma música quando a playlist já estiver cheia, o programa deverá imprimir:

PLAYLIST CHEIA!

Essa verificação deve ser feita antes das demais verificações da função de inserção.

Música repetida

Uma música não pode ser inserida na playlist caso ela já exista. Caso o usuário tente inserir uma música que já esteja na playlist, o programa deverá imprimir:

A MÚSICA JÁ ESTÁ NA PLAYLIST!

Remoção de música inexistente

Uma música não pode ser removida caso ela não exista na playlist. Caso o usuário tente remover uma música que não está na playlist, o programa deverá imprimir:

Remoção bem-sucedida

Sempre que uma música for removida corretamente, o programa deverá imprimir:

MUSICA REMOVIDA: nome da música

Observações importantes

- Os arquivos `.h` disponibilizados não devem ser modificados.
- As funções devem ser implementadas exatamente com as assinaturas definidas nos arquivos `.h`.
- O programa deve respeitar rigorosamente os formatos de entrada e saída.
- As mensagens impressas devem ser exatamente iguais às especificadas no enunciado e nos arquivos de teste.
- Todas as impressões relacionadas à playlist devem utilizar a função `impressFunction`.
- Os ponteiros para função definidos nos cabeçalhos devem ser utilizados.
- Lembre-se de desalocar corretamente toda a memória alocada dinamicamente.
- Utilize os comentários presentes em `musica.h` e `playlist.h` para verificar os formatos esperados de impressão.
- Ponteiros para função podem ser `NULL`.
- Para copiar strings, pode ser útil utilizar a função `strcpy`.